

Gömülü C'ye Sıfırdan

STM32/ESP ile BLDC motor kontrolüne giden yol

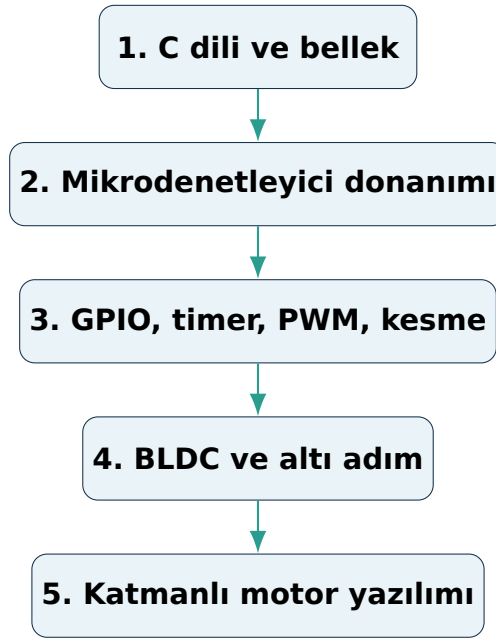
Temelden projeye, sözdiziminden donanıma

Bu kaynak daha önce gömülü C kodu yazmamış bir bilgisayar mühendisliği öğrencisi için hazırlanmıştır. Her kavramda yalnızca “nasıl” değil, “neden” sorusu da cevaplanır.

Hedef proje: Üç fazlı BLDC motoru bir motor sürücü üzerinden altı adımlı komütasyonla güvenli biçimde harekete geçirmek; 60 elektriksel derecelik sektörleri anlamak ve kontrol yazılımını katmanlı kurmak.

Bu kaynak nasıl kullanılmalı?

Bu kitabı baştan sona bir defada ezberlemeye çalışma. Gömülü yazılım, kavramların birbirine bağlandığı bir alandır. Önce küçük bir kodu okuyabilir hale gelir, sonra bir çevre birimini kontrol eder, en sonunda zamanlama ve güvenlik gerektiren motor kontrolüne geçersin.



Her bölümde şu dört kutu türünü göreceksin:

- **Sözdizimi:** Yazım biçimini parçalara ayırır.
- **Neden?:** Bu yapının projede hangi sorunu çözdüğünü açıklar.
- **Dikkat:** Donanım hasarı veya sessiz yazılım hatası oluşturabilecek noktaları belirtir.
- **Bölüm özeti:** Öğrenmiş olman gerekenleri kontrol eder.

Elektriksel güvenlik sınırı

Motorun U, V ve W faz kabloları STM32 veya ESP pinlerine bağlanmaz. Mikrodenetleyici yalnızca sürücü kartının lojik girişlerini kontrol eder. Motor akımı; DC güç kaynağı, inverter MOSFET'leri ve motor fazları üzerinden akar. Gerçek motora enerji vermeden önce sürücünün veri sayfası, besleme gerilimi, akım limiti, dead-time ve hata girişleri doğrulanmalıdır.

İçindekiler

Bu kaynak nasıl kullanılmalı?	i
I C Dilinin Temeli	1
1 Gömülü sistem nedir?	2
1.1 Bilgisayardaki program ile gömülü program arasındaki fark	2
1.2 Bu projedeki fiziksel zincir	2
1.3 Bir mikrodenetleyici programının yaşamı	3
2 Derleme süreci ve ilk C programı	4
2.1 Kaynak kod makine koduna nasıl dönüşür?	4
2.2 İlk programı parçalayalım	4
2.3 Bildirim, ilk değer verme ve atama farklıdır	5
3 Tipler, değişkenler ve operatörler	6
3.1 Neden sabit genişlikli tipler?	6
3.2 İşaretli ve işaretli sayıların tuzağı	6
3.3 Temel operatörler	7
3.4 Tür dönüşümü ve taşıma	7
4 Karar yapıları ve döngüler	8
4.1 if, else if ve else	8
4.2 switch ile durum seçmek	8
4.3 for ve while	9
5 Fonksiyonlar ve programı parçalara ayırmak	10
5.1 Fonksiyon neden vardır?	10
5.2 Değer ile geçirme	10
5.3 Tek sorumluluk ilkesi	11
6 Bellek ve işaretçiler	12
6.1 Değişken aslında nedir?	12
6.2 Adres alma: & operatörü	12
6.3 Adresteki değere erişme: * operatörü	12
6.4 NULL işaretçi	13
6.5 const ile işaretçinin dört hali	13
7 Diziler, tablolar ve sınır güvenliği	15
7.1 Dizi nedir?	15
7.2 Dizi adı ve işaretçi ilişkisi	15
7.3 Tablo tabanlı kontrol	15
7.4 sizeof ile eleman sayısı	16

8 struct, enum ve typedef	17
8.1 enum: sayılara anlamlı ad vermek	17
8.2 struct: ilişkili verileri bir arada tutmak	17
8.3 typedef ne yapar?	17
9 static, const, volatile ve kapsam	19
9.1 Kapsam ve yaşam süresi farklıdır	19
9.2 Dosya seviyesinde static	19
9.3 const: değişmez niyet	19
9.4 volatile: program dışı değişiklik	19
9.5 extern: başka derleme birimindeki sembol	20
10 Header dosyaları ve katmanlı mimari	21
10.1 Bildirim ve tanım	21
10.2 Include guard neden vardır?	21
10.3 Açılı ve tırnaklı include farkı	21
10.4 Bu proje için önerilen katmanlar	22
II Mikrodenetleyici ve Donanımla Konuşmak	23
11 İşlemci, bellek haritası ve register	24
11.1 Mikrodenetleyicinin ana parçaları	24
11.2 Memory-mapped I/O	24
11.3 Read-modify-write işlemi	25
11.4 Maske oluşturmak	25
12 GPIO: dijital giriş ve çıkış	26
12.1 Bir pin neden doğrudan çalışmaz?	26
12.2 Mantıksal seviye fiziksel gerilimdir	26
12.3 Atomik set/reset register'ı	27
13 Timer, PWM ve dead-time	28
13.1 Timer nasıl sayar?	28
13.2 PWM duty nasıl oluşur?	28
13.3 Complementary PWM	28
13.4 Preload ve senkron güncelleme	29
14 Kesme, ISR ve gerçek zaman	30
14.1 Polling ile kesme farkı	30
14.2 İyi ISR nasıl olmalı?	30
14.3 Yarış durumu örneği	31
14.4 Öncelik ve gecikme	31
15 ADC, akım ölçümü ve hata koruması	32
15.1 ADC'nin görevi	32
15.2 PWM ile senkron örnekleme	32
15.3 Yazılım ve donanım koruması birlikte	32
16 Hata ayıklama ve ölçüm araçları	33
16.1 Derleyici uyarıları ilk savunmadır	33
16.2 Debugger ile bakılacaklar	33
16.3 Osiloskop ve logic analyzer	33

III BLDC Motor Kontrolüne Geçiş	35
17 BLDC motorun temel çalışma mantığı	36
17.1 Stator, rotor ve üç faz	36
17.2 Elektriksel ve mekanik açı	36
17.3 Torku ne belirler?	37
18 Üç faz inverter ve altı MOSFET	38
18.1 Bir yarım köprü	38
18.2 Üç yarım köprü bir arada	39
18.3 Üç fazın üç durumu	39
19 Altı adımlı komütasyon	40
19.1 Neden altı adım?	40
19.2 Birinci adımı fiziksel okuyalım	40
19.3 Yön nasıl ters çevrilir?	40
19.4 Komütasyon zamanını kim belirler?	41
20 Hall ve sensorless BEMF	42
20.1 IMU neden gerekli değildir?	42
20.2 Hall kodları	42
20.3 Sadece üç faz kablosu varsa	42
20.4 60° olayından hız hesabı	43
21 Sürücü kartını tanımadan kod yazma	44
21.1 Veri sayfasında aranacak bilgiler	44
22 Motor yazılımı için durum makinesi	45
22.1 Neden tek bir while döngüsü yetmez?	45
22.2 Her durumun sorumluluğu	45
22.3 Durum geçiş koşulları	45
IV Basitten Zora Kodlama Uygulamaları	47
23 Seviye 1: PC'de gate maskesi simülasyonu	48
23.1 Satırların mantığı	49
23.2 Kendin dene	49
24 Seviye 2: Komütasyon tablosu	50
24.1 Faz durumlarını tanımlamak	50
24.2 Altı satırlık değişmez tablo	50
24.3 Bir faz durumunu gate bitine çevirmek	50
25 Seviye 3: Fonksiyon işaretçisiyle donanım soyutlama	52
25.1 Problem	52
25.2 Fonksiyon işaretçisi tipi	52
25.3 PC uygulaması	52
25.4 Kontrol katmanının çağırısı	53
26 Seviye 4: ISR posta kutusu ve olay işleme	54
26.1 Paylaşılan veri yapısı	54
26.2 Erken dönüşler neden kullanıldı?	55
26.3 Bir event bayrağının sınırı	55

27 Seviye 5: Hız hesabı ve hata yolu	56
27.1 İlk ölçüm neden ayrı ele alınır?	56
27.2 Merkezi hata fonksiyonu	57
28 Tam programın çalışma sırası	58
28.1 main fonksiyonunun görevi	58
28.2 Tek Hall olayının yolculuğu	59
28.3 Kodun kavram haritası	59
29 STM32'ye taşıma rehberi	60
29.1 Hangi kod aynı kalır?	60
29.2 Kavramsal STM32 backend	60
29.3 HAL, LL veya register?	61
29.4 ESP32 farkı	61
30 Laboratuvar bring-up planı	62
30.1 Aşama 0: Doküman ve bağlantı	62
30.2 Aşama 1: Motor bağlı değilken	62
30.3 Aşama 2: Düşük enerji	62
30.4 Aşama 3: Kapalı çevrime geçiş	63
31 Sık hatalar ve teşhis yolu	64
32 Mini alıştırmalar	65
Alıştırma 1: Bit maskesi	65
Alıştırma 2: İşaretçi	65
33 Alıştırma çözümleri	66
34 Son kontrol listesi ve çalışma planı	67
34.1 İlk hafta	67
34.2 İkinci hafta	67
34.3 Üçüncü hafta	67
34.4 Dördüncü hafta	67
A Kısa sözlük	69
B Başvuru kaynakları	71

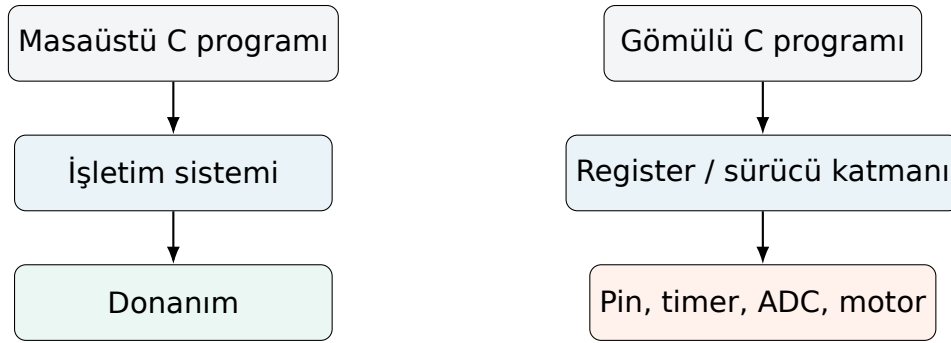
Kısım I

C Dilinin Temeli

Gömülü sistem nedir?

1.1 Bilgisayardaki program ile gömülü program arasındaki fark

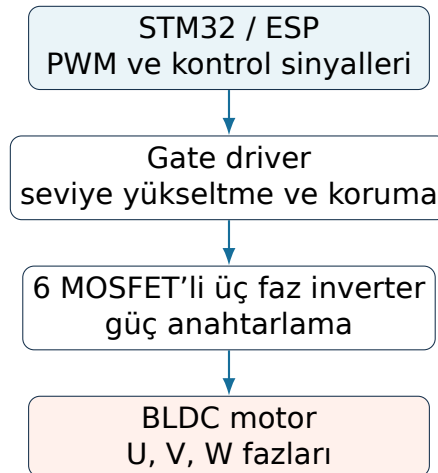
Masaüstü uygulamasında işletim sistemi; ekranı, dosyaları, belleği ve zamanı senin adına yönetir. Mikrodenetleyicide ise program çoğu zaman doğrudan donanım ile konuşur. Bir pine 1 yazdığında fiziksel gerilim değişebilir; yanlış iki MOSFET'i açtığında kısa devre oluşabilir. Bu nedenle gömülü C'de bir satır kodun hem yazılımsal hem elektriksel karşılığı vardır.



Şekil 1.1: Masaüstü ve yalın gömülü programın sadeleştirilmiş farkı

1.2 Bu projedeki fiziksel zincir

BLDC projesinde bağlantı zinciri şöyledir:



Neden?

Bu zinciri anlamadan kod yazmak tehlikelidir. Mikrodenetleyici motor akımını taşımaz; yalnızca güç katmanının ne zaman anahtarlanaacağını söyler. Bu yüzden “motor kablosunu STM’ye bağlarım” düşüncesi yanlıştır.

1.3 Bir mikrodenetleyici programının yaşamı

Enerji verildiğinde işlemci reset vektöründen başlar, başlangıç kodu RAM’i hazırlar ve sonra main() fonksiyonuna gelir. Basit bir yalın-metal program çoğunlukla hiç bitmeyen döngü içerir:

Listing 1.1: Bir gömülü programın en sade iskeleti

```
1 #include <stdint.h>
2
3 int main(void)
4 {
5     donanimi_hazirla();
6
7     while (1)
8     {
9         olaylari_isle();
10    }
11 }
```

Sözdizimi

int main(void) ifadesinde int, fonksiyonun dönüş tipidir; main fonksiyonun adıdır; parantez içindeki void, parametre almadığını açıkça belirtir. Süslü parantezler fonksiyon gövdesini sınırlar. Her ifade çoğunlukla noktalı virgülle biter.

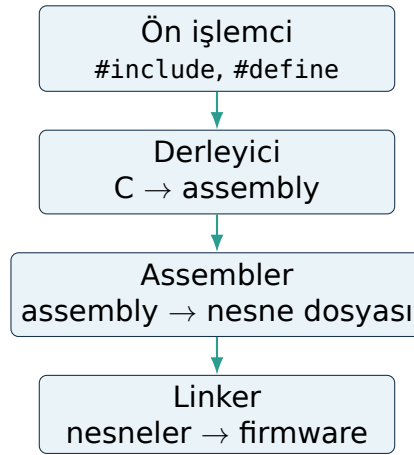
Bölüm özeti

- Mikrodenetleyici kodu fiziksel donanım durumunu değiştirebilir.
- MCU, gate driver ve inverter aynı şey değildir.
- Yalın-metal uygulamada while (1) ana çalışma döngüsüdür.

Derleme süreci ve ilk C programı

2.1 Kaynak kod makine koduna nasıl dönüşür?

Yazdığın .c dosyası işlemcinin doğrudan anlayacağı biçimde değildir. Araç zinciri dört temel aşamada çalışır:



Şekil 2.1: C kaynak kodunun firmware'e dönüşmesi

Neden?

"Undefined reference" hatası genellikle sözdizimi hatası değil, linker hatasıdır: fonksiyon bildirilmiştir ama tanımı bağlanan dosyalarda yoktur. "Unknown type name" veya eksik noktalı virgül ise çoğunlukla derleyici aşamasında yakalanır.

2.2 İlk programı parçalayalım

Listing 2.1: Değişken, işlem ve fonksiyon çağırısı

```

1 #include <stdint.h>
2 #include <stdio.h>
3
4 int main(void)
5 {
6     uint32_t sayac = 0u;
7
8     sayac = sayac + 1u;
9     printf("sayac = %u\n", (unsigned)sayac);
10

```

```

11     return 0;
12 }

```

Parça	Anlamı
#include <stdint.h>	Sabit genişlikli tamsayı tiplerinin bildirimlerini ekler.
uint32_t	Tam 32 bitlik işaretli tamsayı tipidir.
sayac	Değişkenin programcı tarafından seçilen adıdır.
= 0u	İlk değer atamasıdır. u, sabitin işaretli olduğunu belirtir.
sayac + 1u	Mevcut değer okunur, bir eklenir.
printf(...)	PC testinde ekrana yazar; yalın MCU'da kendiliğinden ekran oluşturmaz.
return 0;	Programın başarılı bittiğini bildirir; gömülü ana döngüde çoğu zaman bu satıra ulaşmaz.

2.3 Bildirim, ilk değer verme ve atama farklıdır

```

1 uint32_t hiz;          /* Bildirim: hiz 1adnda alan 1ayr. */
2 uint32_t hedef = 0;    /* Bildirim + ilk değeer verme. */
3
4 hiz = 1200;            /* Atama: var olan değdeikeni değdeitir. */

```

Dikkat

Otomatik yerel değişken ilk değer verilmeden okunursa belirsiz değer içerir. Derleyici uyarılarını açmak bu tür hataların önemli bir bölümünü erkenden yakalar: -Wall -Wextra -Wpedantic.

Tipler, değişkenler ve operatörler

3.1 Neden sabit genişlikli tipler?

C standardında `int` tipinin kaç bit olduğu hedefe göre değişebilir. Register ve haberleşme paketi işlerken genişliğin kesin olması gerekir. Bu yüzden `stdint.h` tipleri tercih edilir.

Tip	Yaklaşık aralık	Projede kullanım
<code>uint8_t</code>	0 ... 255	Hall kodu, byte, küçük durum değeri
<code>int8_t</code>	-128 ... 127	-1 gibi geçersiz tablo değeri
<code>uint16_t</code>	0 ... 65535	ADC sonucu, küçük timer
<code>uint32_t</code>	0 ... $2^{32} - 1$	Register, zaman sayacı, RPM
<code>int32_t</code>	$-2^{31} ... 2^{31} - 1$	İşaretli hata ve fark hesabı
<code>uint64_t</code>	Çok geniş	Ara hesapta taşmayı önleme
<code>float</code>	Yaklaşık 7 basamak	Fiziksel büyüklük, gerektiğinde

Neden her yerde float kullanmıyoruz?

Bazı mikrodenetleyiciler kayan nokta işlemlerini yavaş yapar; ayrıca karşılaştırma ve gerçek zamanlı süre açısından tamsayı daha öngörülebilir olabilir. Örneğin duty değerini yüzde yerine 0-1000 arası tamsayıyla tutmak mümkündür. Yine de ölçüm ve kontrol algoritmasında gerektiğinde `float` kullanmak yanlış değildir; hedef işlemci ve süre bütçesi belirleyicidir.

3.2 İşaretli ve işaretsiz sayıların tuzağı

```
1 uint32_t a = 0u;
2 a = a - 1u; /* Sonuç -1 ğdeil, 4294967295 olur. */
```

İşaretsiz aritmetik modüler çalışır. Bu özellik timer taşmasını hesaplamak için faydalı olabilir:

```
1 uint32_t gecen = simdiki_tick - onceki_tick;
```

Sayaç bir kez taşsa bile iki değer arasındaki süre, uygun aralıkta kaldığı sürece doğru hesaplanır.

3.3 Temel operatörler

Grup	Operatörler	Örnek anlam
Aritmetik	+ - * / %	Toplama, çıkarma, çarpma, bölme, kalan
Karşılaştırma	== != < > < = > =	İki değeri karşılaştırır; sonuç doğru/yanlış
Mantıksal	&& !	Koşulları birleştirir
Bit düzeyi	& ^ ~ << >>	Register bitlerini değiştirir
Atama	= + = - = & = =	Değişkene yeni değer yazar
Adres	& *	Adres alma ve işaret edilen değere erişim
Üye erişimi	. ->	Yapı nesnesi veya yapı işaretçisi üyesi

En sık başlangıç hatası

= atama yapar, == karşılaştırır. `if (state = MOTOR_RUN)` yazmak `state`'i değiştirir ve çoğu zaman koşulu doğru yapar. Doğrusu `if (state == MOTOR_RUN)` biçimidir.

3.4 Tür dönüşümü ve taşıma

RPM hesabında önce çarpma yapılırsa 32 bit ara değer taşabilir. Bu yüzden en az bir terim 64 bit yapılır:

```
1 uint64_t pay = 60ull * timer_frequency_hz;
2 uint32_t rpm = (uint32_t)(pay / payda);
```

Parantez içindeki `uint32_t` açık tür dönüşümüdür. Sonucun bu tipe sığdığına programcı karar vermiş olur.

Karar yapıları ve döngüler

4.1 if, else if ve else

```
1 if (akim_ma > akim_limiti_ma)
2 {
3     motoru_kapat();
4 }
5 else if (baslat_komutu != 0u)
6 {
7     motoru_baslat();
8 }
9 else
10 {
11     motoru_beklet();
12 }
```

Koşul sırası önemlidir. Güvenlik kontrolü başlatma komutundan sonra olsaydı motor kısa süre de olsa tehlikeli durumda çalıştırılabilirdi.

4.2 switch ile durum seçmek

```
1 switch (motor_state)
2 {
3     case MOTOR_IDLE:
4         /* Bekle. */
5         break;
6
7     case MOTOR_RUN:
8         motoru_islet();
9         break;
10
11    case MOTOR_FAULT:
12        gate_cikislarini_kapat();
13        break;
14
15    default:
16        motor_state = MOTOR_FAULT;
17        break;
18 }
```

Sözdizimi

case belirli bir değere karşılık gelen dalı seçer. break, o daldan çıkmayı sağlar. Unutulursa yürütme sonraki case'e düşer. Bazen bu bilerek yapılır; başlangıçta ise çoğunlukla hatadır.

4.3 for ve while

```
1 for (uint32_t i = 0u; i < 6u; ++i)
2 {
3     tabloyu_kontrol_et(i);
4 }
5
6 while (1)
7 {
8     motor_control_process();
9 }
```

for üç parçadan oluşur: başlangıç, devam koşulu ve her tur sonundaki değişiklik. while ise yalnızca koşula sahiptir.

Bloklayan gecikme

Uzun delay() çağrıları başlangıç denemesinde kolaydır fakat motor kontrolünde ana döngünün hata ve sensör olaylarını zamanında işlemesini engelleyebilir. Gerçek zamanlı projede timer, durum makinesi ve kesme tabanlı zamanlama tercih edilir.

Fonksiyonlar ve programı parçalara ayırmak

5.1 Fonksiyon neden vardır?

Fonksiyon, tek bir görevi isimlendirir. Aynı işlemi kopyalamayı önler, test edilebilirliği artırır ve ana akışı okunur hale getirir.

```

1 static uint32_t rpm_hesapla(
2     uint32_t timer_hz,
3     uint32_t sector_ticks,
4     uint8_t pole_pairs)
5 {
6     if ((sector_ticks == 0u) || (pole_pairs == 0u))
7     {
8         return 0u;
9     }
10
11     uint64_t pay = 60ull * timer_hz;
12     uint64_t payda = 6ull * pole_pairs * sector_ticks;
13     return (uint32_t)(pay / payda);
14 }
```

Fonksiyon imzasını oku

static fonksiyonun bu kaynak dosyaya özel olduğunu; uint32_t geri dönüş tipini; rpm_hesapla adını; parantez içindekiler ise parametreleri belirtir. return, sonucu çağırana koda döndürür.

5.2 Değer ile geçirme

C'de parametreler varsayılan olarak kopyalanır:

```

1 void bir_artir(uint32_t x)
2 {
3     x = x + 1u;
4 }
5
6 uint32_t sayi = 5u;
7 bir_artir(sayi); /* sayi hala 5 */
```

Çağıranın değişkenini değiştirmek için adresi gönderilir; bu konu işaretçiler bölümünde açıklanacaktır.

5.3 Tek sorumluluk ilkesi

İyi bir motor kontrol modülünde görevler ayrılır:

- `read_sensors()`: ölçümleri alır.
- `select_step()`: komütasyon adımını seçer.
- `write_gates()`: donanıma uygular.
- `enter_fault()`: güvenli hata durumuna geçirir.

Bu ayırım sayesinde hesaplama kodu PC'de test edilirken gerçek MOSFET'lere enerji verilmez.

Bellek ve işaretçiler

6.1 Değişken aslında nedir?

Bir değişken yalnızca isim değildir; bellekte belirli sayıda byte kaplayan bir bölgedir. İsim, derleme sırasında bu bölgeye erişmeyi kolaylaştırır. İşlemci çalışırken asıl önemli olan adrestir.

Adres	Bellekteki değer
0x20000020	0x78
0x20000021	0x56
0x20000022	0x34
0x20000023	0x12

uint32_t sayi = 0x12345678

Şekil 6.1: 32 bit bir değişkenin little-endian bellekte örnek görünümü

6.2 Adres alma: & operatörü

```
1 uint32_t hiz = 1200u;
2 uint32_t *hiz_adresi = &hiz;
```

Satırları parçalara ayıralım:

- `uint32_t *`: 32 bitlik işaretsiz bir değerın adresini tutabilen tip.
- `hiz_adresi`: İşaretçi değişkeninin adı.
- `&hiz`: `hiz` değişkeninin bellekteki adresi.

6.3 Adresteki değere erişme: * operatörü

```
1 *hiz_adresi = 1500u;
```

Bu satır “`hiz_adresi`’nin gösterdiği yere 1500 yaz” demektir. Sonuç olarak `hiz` değişkeni 1500 olur.



İşaretçi de bir değişkendir.

Gösterilen nesne

Motor kodunda işaretçi neden gerekli?

`motor_control_start(&motor)` çağrısında yapının kopyası değil adresi gönderilir. Fonksiyon aynı motor nesnesinin durumunu değiştirir. Büyük yapıların kopyalanması önlenir ve donanım register adreslerine erişmek mümkün olur.

6.4 NULL işaretçi

NULL, geçerli bir nesneyi göstermeyen işaretçiye temsil eder:

```

1 void motor_start(MotorControl *motor)
2 {
3     if (motor == NULL)
4     {
5         return;
6     }
7
8     motor->state = MOTOR_RUN;
9 }
  
```

`motor->state`, şu ifadenin daha kısa yazımıdır:

```

1 (*motor).state = MOTOR_RUN;
  
```

Önce `*motor` ile yapı bulunur, sonra `.state` üyesine erişilir.

Dikkat

NULL veya geçersiz bir adresi `*` ile açmak tanımsız davranıştır. PC'de program çökebilir; mikrodenetleyicide HardFault oluşabilir veya daha kötüsü yanlış çevre birimi değişebilir.

6.5 const ile işaretçinin dört hali

Bildirim	Anlam
<code>uint32_t *p</code>	Değer ve işaretçinin yönü değişebilir.
<code>const uint32_t *p</code>	Gösterilen değer bu işaretçi üzerinden değiştirilemez; işaretçi başka yere yönlendirilebilir.
<code>uint32_t *const p</code>	İşaretçi başka adrese yönlendirilemez; gösterilen değer değiştirilebilir.
<code>const uint32_t *const p</code>	Ne adres ne de değer bu isim üzerinden değiştirilebilir.

Donanım örneğinde sık görülen biçim:

```
1 static volatile uint32_t *const gate_register =  
2   (volatile uint32_t *)0x40010000u;
```

Burada işaretçi hep aynı donanım adresini gösterir, fakat adresteki register değeri değişebilir.

Diziler, tablolar ve sınır güvenliği

7.1 Dizi nedir?

Dizi aynı tipte art arda yerleştirilmiş nesneler topluluğudur:

```
1 uint8_t hall_sequence[6] =
2 {
3     0x01u, 0x05u, 0x04u,
4     0x06u, 0x02u, 0x03u
5 };
```

Geçerli indeksler 0 ile 5 arasındadır. Altı elemanlı dizide `hall_sequence[6]` yoktur.

Sınır dışına yazma

C dizinin sınırını çalışma anında otomatik kontrol etmez. Sınır dışı yazma başka değişkeni, stack'i veya dönüş adresini bozabilir. İndeksin her zaman tablo boyutundan küçük olduğunu doğrula.

7.2 Dizi adı ve işaretçi ilişkisi

Çoğu ifadeye dizi adı ilk elemana işaretçiye dönüşür:

```
1 uint8_t values[3] = {10u, 20u, 30u};
2 uint8_t *p = values;
3
4 uint8_t a = values[1]; /* 20 */
5 uint8_t b = *(p + 1); /* 20 */
```

İşaretçi aritmetiğinde `p + 1`, bir byte değil, gösterilen tipin bir elemanı kadar ilerler.

7.3 Tablo tabanlı kontrol

Altı adımı altı ayrı if ile yazmak mümkündür; ancak tablo daha açık ve denetlenebilirdir:

```
1 static const int8_t hall_to_step[8] =
2 {
3     -1, 0, 4, 5, 2, 1, 3, -1
4 };
5
6 int8_t step = hall_to_step[hall_code];
```

Neden?

Hall kodu 3 bit olduğu için 0-7 arasındır; bu yüzden sekiz elemanlı tablo doğal bir eşlemedir. -1, 000 ve 111 gibi geçersiz girişleri ifade eder. Tablo, kablo sırası değiştiğinde kontrol akışını yeniden yazmadan eşlemeyi değiştirmeyi sağlar.

7.4 sizeof ile eleman sayısı

```
1 #define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]))
2
3 for (size_t i = 0u; i < ARRAY_LEN(hall_sequence); ++i)
4 {
5     /* ... */
6 }
```

Bu makro yalnızca gerçek dizi aynı kapsamdayken güvenlidir. Fonksiyon parametresine geçen dizi işaretçiye dönüştüğü için orada toplam boyut kaybolur.

struct, enum ve typedef

8.1 enum: sayılara anlamlı ad vermek

```
1 typedef enum
2 {
3     MOTOR_IDLE,
4     MOTOR_ALIGN,
5     MOTOR_STARTUP,
6     MOTOR_RUN,
7     MOTOR_FAULT
8 } MotorState;
```

Derleyici arka planda tamsayı kullanır, fakat kodda 3 yerine MOTOR_RUN görülür. Bu hata olasılığını azaltır.

8.2 struct: ilişkili verileri bir arada tutmak

```
1 typedef struct
2 {
3     MotorState state;
4     uint8_t current_step;
5     uint16_t duty_per_mille;
6     uint32_t measured_rpm;
7     uint32_t fault_flags;
8 } MotorControl;
```

Bir nesne oluşturmak:

```
1 MotorControl motor =
2 {
3     .state = MOTOR_IDLE,
4     .current_step = 0u,
5     .duty_per_mille = 0u,
6     .measured_rpm = 0u,
7     .fault_flags = 0u
8 };
```

Noktalı ilk değer verme, hangi değerın hangi üyeye gittiğini açıkça gösterir.

8.3 typedef ne yapar?

typedef yeni bir tip oluşturmaz; mevcut tipe okunabilir bir takma ad verir:

```
1 typedef uint32_t Tick;  
2 typedef void (*FaultCallback)(uint32_t flags);
```

İkinci satır bir fonksiyon işaretçisi tipine ad verir. Bu konu donanım soyutlama bölümünde kullanılacaktır.

Neden?

Motorun state, RPM ve fault değerlerini ayrı global değişkenler yapmak yerine tek MotorControl yapısında tutmak; birden fazla motor eklemeyi, fonksiyonlara bağlam göndermeyi ve test yazmayı kolaylaştırır.

static, const, volatile ve kapsam

9.1 Kapsam ve yaşam süresi farklıdır

Kapsam, bir isme kodun nerelerinden erişilebildiğini; **yaşam süresi**, nesnenin bellekte ne kadar kaldığını anlatır.

```
1 static uint32_t module_counter; /* Dosya 1kapsam, program boyu şyaam. */
2
3 static void helper(void)      /* 1Yalnız bu .c 1dosyasından çağılır. */
4 {
5     static uint32_t call_count; /* Yerel kapsam, program boyu şyaam. */
6     uint32_t temporary = 0u;    /* Blok 1kapsam, çağır süresince şyaam. */
7
8     ++call_count;
9     ++temporary;
10 }
```

9.2 Dosya seviyesinde static

static bir global değişken veya fonksiyonda kullanılırsa sembolü dosya dışına kapatır. Bu, modülün iç ayrıntılarını saklar:

```
1 static void enter_fault(MotorControl *motor)
2 {
3     motor->state = MOTOR_FAULT;
4 }
```

Başka dosyaların yalnızca public fonksiyonları kullanması, bağımlılığı azaltır.

9.3 const: değişmez niyet

```
1 static const CommutationStep forward_table[6] =
2 {
3     /* ... */
4 };
```

const, tablonun kazara değiştirilmesini derleme hatasına dönüştürür. Gömülü linker çoğunlukla bunu Flash'taki salt okunur bölüme koyar.

9.4 volatile: program dışı değişiklik

```
1 static volatile uint8_t event_pending;
2
3 void hall_isr(void)
4 {
5     event_pending = 1u;
6 }
7
8 int main(void)
9 {
10     while (1)
11     {
12         if (event_pending != 0u)
13         {
14             event_pending = 0u;
15             process_event();
16         }
17     }
18 }
```

Derleyici, normal akışta değişmediğini düşündüğü bir değeri tekrar okumayabilir. Kesme, DMA veya donanım register'ı bu değeri dışarıdan değiştirebildiği için volatile her erişimin gerçekten yapılmasını ister.

volatile ne sağlamaz?

volatile; atomiklik, karşılıklı dışlama, işlem sırası kilidi veya çok alanlı tutarlı anlık görüntü sağlamaz. ISR üç değişkeni güncellerken ana döngü bunların yarısını eski, yarısını yeni okuyabilir. Kritik bölüm, atomik protokol veya kuyruk gerekebilir.

9.5 extern: başka derleme birimindeki sembol

Header'da:

```
1 extern volatile uint32_t system_ticks;
```

Yalnız bir .c dosyasında tanım:

```
1 volatile uint32_t system_ticks = 0u;
```

Her header'a doğrudan tanım yazmak linker'da çoklu tanım hatası oluşturabilir. Yine de mümkün olduğunca global extern yerine erişim fonksiyonları ve bağlam yapıları tercih edilir.

Header dosyaları ve katmanlı mimari

10.1 Bildirim ve tanım

- **Bildirim:** Bir isim ve tipin var olduğunu söyler.
- **Tanım:** Fonksiyon gövdesini veya nesnenin gerçek depolamasını oluşturur.

Listing 10.1: motor_control.h içinde public bildirim

```
1 #ifndef MOTOR_CONTROL_H
2 #define MOTOR_CONTROL_H
3
4 #include <stdint.h>
5
6 typedef struct MotorControl MotorControl;
7
8 void motor_control_init(MotorControl *motor);
9 void motor_control_start(MotorControl *motor);
10 void motor_control_process(MotorControl *motor);
11
12 #endif
```

Listing 10.2: motor_control.c içinde tanım

```
1 #include "motor_control.h"
2
3 void motor_control_start(MotorControl *motor)
4 {
5     /* Fonksiyonun gerçek gövdesi. */
6 }
```

10.2 Include guard neden vardır?

Bir header dolaylı olarak birkaç kez eklenebilir. `#ifndef`, `#define` ve `#endif` üçlüsü içeriğin derleme biriminde yalnız bir kez işlenmesini sağlar.

10.3 Açılı ve tırnaklı include farkı

```
1 #include <stdint.h>           /* Sistem/standart header. */
2 #include "motor_control.h"    /* Projeye ait header. */
```

Arama yollarının kesin davranışı araç zincirine bağlıdır; genel niyet bu ayrımdır.

10.4 Bu proje için önerilen katmanlar

Uygulama: başlat/durdur, hedef hız, telemetri

Kontrol: state machine, komütasyon, hız hesabı

Donanım soyutlama: PWM, capture, ADC, fault

STM32/ESP çevre birimleri ve motor sürücü

Neden?

Kontrol algoritmasının içinde doğrudan belirli STM32 register'ı kullanılırsa aynı algoritmayı PC'de test etmek veya ESP'ye taşımak zorlaşır. Donanım katmanı değişir, kontrol mantığı mümkün olduğunca değişmeden kalır.

Birinci kısım kontrol listesi

Şunları açıklayabiliyorsan mikrodenetleyici bölümüne geçebilirsin:

- `uint32_t *p = &x;` satırında `*` ve `&` ne yapar?
- `motor->state` neden nokta yerine ok operatörü kullanır?
- `static const` bir komütasyon tablosuna ne kazandırır?
- ISR'in değiştirdiği bayrak neden `volatile` olur ve bu neden tek başına yeterli değildir?
- Header ile `.c` dosyasının sorumluluğu nedir?

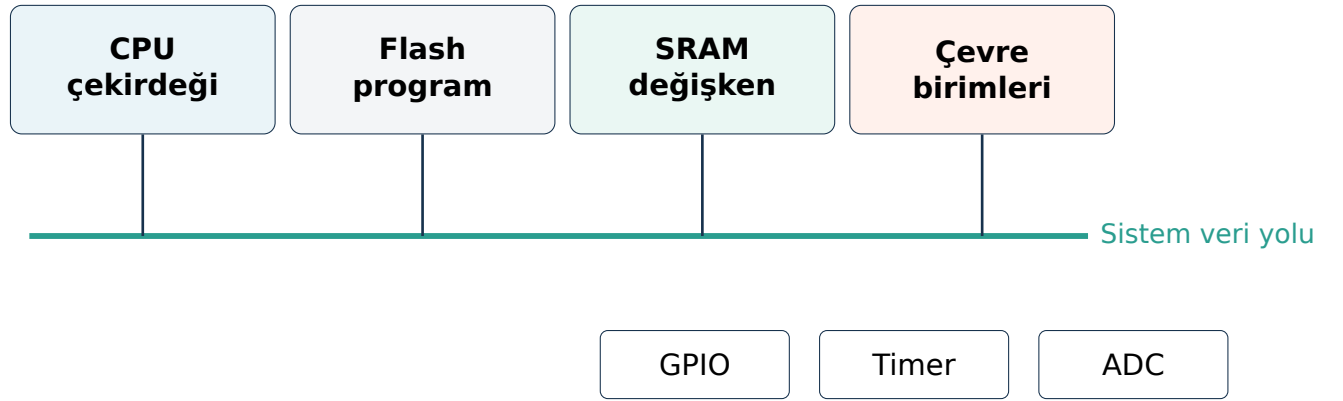
Kısım II

Mikrodenetleyici ve Donanımla Konuşmak

İşlemci, bellek haritası ve register

11.1 Mikrodenetleyicinin ana parçaları

Bir STM32 veya ESP yalnızca CPU değildir. Aynı yonga içinde program belleği, RAM ve çevre birimleri bulunur.



Şekil 11.1: Mikrodenetleyicinin kavramsal iç yapısı

11.2 Memory-mapped I/O

Çevre birimi register'ları bellek adresi uzayına yerleştirilir. CPU açısından RAM okumaya benzeyen bir yükleme işlemi, timer sayacını okuyabilir; bir yazma işlemi GPIO pinini değiştirebilir.

Adres bölgesi	Kavramsal içerik	Örnek kullanım
Flash bölgesi	Program ve sabit veri	Komütasyon tablosu
SRAM bölgesi	Çalışma verisi	Motor state, ölçümler
Peripheral bölgesi	Donanım register'ları	GPIO, TIM, ADC

Gerçek adresleri elle ezberlemek yerine üreticinin CMSIS header'ları kullanılır:

```

1 TIM1->CCR1 = duty_ticks;
2 GPIOA->BSRR = (1u << 5);

```

TIM1->CCR1 nasıl okunur?

TIM1, timer register yapısına işaretçidir. ->CCR1, bu yapının Capture/Compare Register 1 üyesini seçer. Atama, bu donanım register'ına değer yazar.

11.3 Read-modify-write işlemi

Bir register'ın yalnız belirli bitini değiştirmek için önce mevcut değer okunabilir:

```
1 register_value |= bit_mask; /* Bitleri 1 yap. */
2 register_value &= ~bit_mask; /* Bitleri 0 yap. */
3 register_value ^= bit_mask; /* Bitleri tersle. */
```

Fakat donanım register'larında bu yöntem her zaman güvenli değildir. Bazı bitler "1 yazarak temizle", bazıları yalnız okunur olabilir. Ayrıca ISR aynı anda register'ı değiştirirse güncelleme kaybolabilir. Referans kılavuzundaki register davranışı okunmalıdır.

11.4 Maske oluşturmak

```
1 #define BIT(n) (1u << (n))
2
3 uint32_t mask = BIT(0) | BIT(3);
```

İki bit alanına değer yerleştirmek:

```
1 #define MODE_Pos 4u
2 #define MODE_Msk (0x3u << MODE_Pos)
3
4 reg = (reg & ~MODE_Msk) | ((new_mode << MODE_Pos) & MODE_Msk);
```

Adımlar:

1. `reg & ~MODE_Msk`: Eski alanı temizler.
2. `new_mode << MODE_Pos`: Yeni değeri doğru bit konumuna kaydırır.
3. İkinci maske, komşu bitlere taşmayı engeller.
4. `|` yeni alanı register'a ekler.

GPIO: dijital giriş ve çıkış

12.1 Bir pin neden doğrudan çalışmaz?

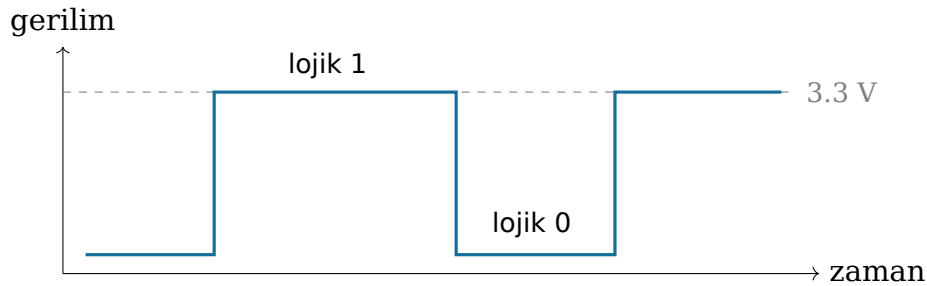
Mikrodenetleyici pinleri çok amaçlıdır. Bir pini kullanmadan önce genellikle şu ayarlar yapılır:

1. İlgili GPIO portunun clock'u açılır.
2. Pin modu input, output veya alternate function seçilir.
3. Çıkış tipi push-pull/open-drain belirlenir.
4. Hız ve pull-up/pull-down ayarlanır.
5. Gerekirse alternate function numarası seçilir.

Clock neden açılıyor?

Çevre birimleri kullanılmadığında clock'ları kapatılarak güç tüketimi azaltılır. Clock kapalıyken register yazısı etkisiz kalabilir. Başlangıçta "kodu yazdım ama pin değişmiyor" sorununun yaygın sebeplerinden biridir.

12.2 Mantıksal seviye fiziksel gerilimdir



Şekil 12.1: Bir dijital çıkışın gerilim karşılığı

MCU çıkışı gate driver'ın lojik girişine gidebilir; doğrudan MOSFET gate'ini sürmek bile çoğu güç uygulamasında uygun değildir. Gate'in hızlı şarj/deşarjı, high-side seviye yükseltme ve koruma için sürücü gerekir.

12.3 Atomik set/reset register'ı

STM32 ailesindeki birçok GPIO, BSRR benzeri set/reset register'ı sunar. Böylece read-modify-write yarışından kaçınılır:

```
1 GPIOA->BSRR = (1u << 5);          /* PA5 set. */  
2 GPIOA->BSRR = (1u << (5u+16u)); /* PA5 reset. */
```

Kesin register adı ve davranışı seçilen MCU'nun referans kılavuzundan doğrulanmalıdır.

Timer, PWM ve dead-time

13.1 Timer nasıl sayar?

Timer giriş clock'unu prescaler ile böler, sayaç 0'dan ARR değerine kadar ilerler. Basitleştirilmiş güncelleme frekansı:

$$f_{update} = \frac{f_{timer}}{(PSC + 1)(ARR + 1)}$$

Örnek olarak timer clock'u 80 MHz, PSC =0 ve ARR =3999 ise:

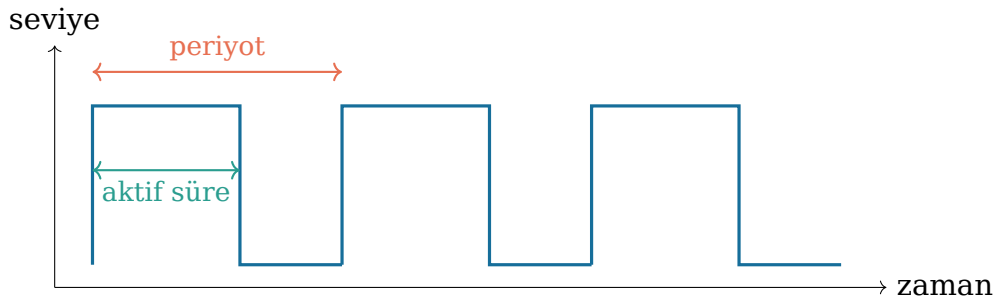
$$f_{PWM} = \frac{80\,000\,000}{1 \cdot 4000} = 20\,000 \text{ Hz}$$

Gerçek STM32 timer clock hesabında APB prescaler etkisi olabileceği için clock tree kontrol edilmelidir.

13.2 PWM duty nasıl oluşur?

Edge-aligned PWM'de sayaç CCR değerinden küçükken çıkış aktif seçilebilir:

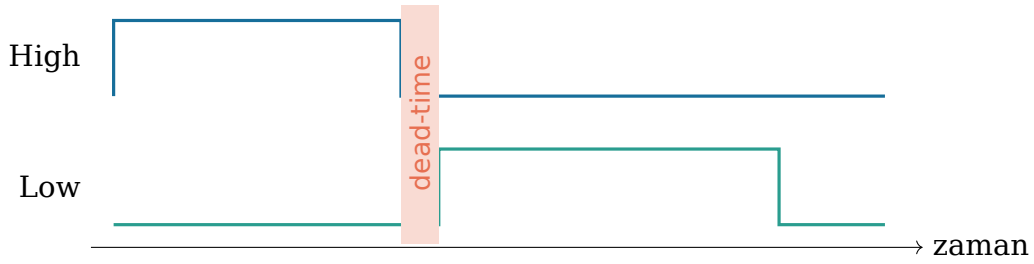
$$D = \frac{CCR}{ARR + 1}$$



Şekil 13.1: Yaklaşık yüzde 60 duty değerli edge-aligned PWM

13.3 Complementary PWM

Bir yarım köprüde high-side ve low-side çıkışlar tamamlayıcı olabilir. Fakat biri kapanır kapanmaz diğeri açmak tehlikelidir; MOSFET'in kapanma süresi sıfır değildir.



Şekil 13.2: İki anahtarın kapalı tutulduğu dead-time aralığı

Shoot-through

Aynı fazın high-side ve low-side MOSFET'i birlikte açılırsa DC+ ile GND arasında çok düşük dirençli yol oluşur. Buna shoot-through denir. Yazılım maskesi hatalı deseni engelleyebilir, fakat geçiş anındaki gecikmeyi yalnız donanımsal dead-time ve sürücü interlock'u güvenilir biçimde yönetir.

13.4 Preload ve senkron güncelleme

Üç fazın compare değerlerini farklı anlarda etkinleştirmek kısa süreli yanlış anahtar deseni oluşturabilir. Gelişmiş timer'larda preload register'larına yazılır ve update/commutation olayıyla bütün yeni değerler aynı sınırdan etkinleştirilir.

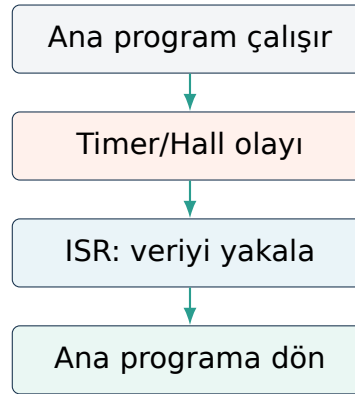
Motor kontrolünde timer seçerken aranan özellikler:

- Complementary çıkışlar
- Programlanabilir dead-time
- Break/fault girişi
- Preload ve senkron update
- Center-aligned PWM seçeneği
- ADC tetikleme olayı

Kesme, ISR ve gerçek zaman

14.1 Polling ile kesme farkı

Polling'de ana döngü sürekli bir bayrağı sorar. Kesmede donanım olay olduğunda CPU'yu ISR'a yönlendirir.



14.2 İyi ISR nasıl olmalı?

ISR mümkün olduğunca kısa ve süre olarak öngörülebilir olmalıdır:

```

1 void HALL_IRQHandler(void)
2 {
3     uint8_t hall = read_hall_pins();
4     uint32_t capture = read_capture_timer();
5
6     hall_mailbox.code = hall;
7     hall_mailbox.capture = capture;
8     hall_mailbox.pending = 1u;
9
10    clear_interrupt_flag();
11 }
  
```

ISR içinde kaçınılması gerekenler:

- Uzun bloklayan döngü veya gecikme
- Dinamik bellek ayırma
- Yavaş printf çağrısı
- Sonlanma süresi belirsiz haberleşme
- Gereksiz ağır kayan nokta hesabı

14.3 Yarış durumu örneği

Ana döngü capture değerinin ilk yarısını okurken ISR yeni değer yazarsa tutarsız veri oluşabilir. 32 bit Cortex-M üzerinde hizalı 32 bit tek erişim genellikle atomik olsa bile birden fazla alanın birlikte tutarlı olması garanti değildir.

Basit kritik bölüm:

```
1 disable_interrupts();
2 local_code = mailbox.code;
3 local_capture = mailbox.capture;
4 mailbox.pending = 0u;
5 enable_interrupts();
```

Kritik bölüm çok kısa tutulmalıdır; aksi halde başka önemli kesmeler gecikir.

14.4 Öncelik ve gecikme

Motor fault kesmesi, kullanıcı butonu kesmesinden daha kritik olabilir. Ancak yalnızca öncelik numarasını değiştirmek yeterli değildir; seçilen MCU'da öncelik bitlerinin ve RTOS kurallarının nasıl çalıştığı doğrulanmalıdır.

ADC, akım ölçümü ve hata koruması

15.1 ADC'nin görevi

ADC analog gerilimi sayısal değere çevirir. 12 bit ADC ve 3.3 V referans için ideal yaklaşım:

$$V_{in} = \frac{ADC_{raw}}{4095} V_{ref}$$

Akım şönt üzerinden ölçülüyorsa:

$$I = \frac{V_{shunt}}{R_{shunt} G}$$

Burada G , akım yükseltecinin kazancıdır.

15.2 PWM ile senkron örnekleme

Anahtarlama kenarları elektriksel gürültü üretir. ADC örneğini PWM periyodunun sakın bir noktasında timer ile tetiklemek daha güvenilir sonuç verir. Rastgele anda yazılım çağrısıyla okumak, komütasyon gürültüsünü ölçüme karıştırabilir.

15.3 Yazılım ve donanım koruması birlikte

Koruma	Donanım yolu	Yazılım yolu
Aşırı akım	Comparator → gate driver / timer break	ADC ölçümü, fault kaydı
Düşük gerilim	Sürücü UVLO	ADC ile DC bus izleme
Aşırı sıcaklık	Driver shutdown / sensör	Sıcaklık eşiği ve azaltma
Geçersiz sektör	Gate interlock	Hall/BEMF doğrulama
Watchdog	Bağımsız watchdog reset	Periyodik sağlıklı çalışma işareti

Dikkat

Yazılımın ADC okuyup motoru kapatmasını tek aşırı akım koruması yapmak yavaştır. Şiddetli kısa devrede donanımsal comparator ve gate shutdown yolu, CPU'dan bağımsız ve çok daha hızlı olmalıdır.

Hata ayıklama ve ölçüm araçları

16.1 Derleyici uyarıları ilk savunmadır

PC testlerinde örnek bayraklar:

```
1 gcc -std=c11 -Wall -Wextra -Wpedantic \
2   main.c motor_control.c motor_hw.c -o motor_test
```

Gömülü projede de IDE'nin uyarı seviyesini yüksek tut. Uyarıyı susturmak yerine sebebini anlamaya çalış.

16.2 Debugger ile bakılacaklar

- Breakpoint: Programı seçilen satırda durdurur.
- Step over: Fonksiyonu tek adım olarak yürütür.
- Step into: Fonksiyonun içine girer.
- Watch: Değişkeni sürekli izler.
- Peripheral view: Register'ları isimleriyle gösterir.
- Call stack: Fonksiyon çağrı zincirini gösterir.

16.3 Osiloskop ve logic analyzer

Motor projesinde yazılım debugger'ı tek başına yeterli değildir. PWM frekansı, complementary çıkış, dead-time ve sektör sırası osiloskopa doğrulanır. Hall veya dijital sürücü sinyalleri logic analyzer ile daha uzun süre kaydedilebilir.

Ölçüm toprağı

Osiloskop probunun toprak klipsi birçok masaüstü osiloskopta koruyucu toprağa bağlıdır. Yanlış noktaya bağlamak kısa devre oluşturabilir. Güç katmanı ölçümünde diferansiyel prob, izolasyon ve kurum laboratuvar prosedürü gerekir.

Donanım kısmı kontrol listesi

- Register'ın bellek adresinden erişilen donanım durumu olduğunu açıklayabiliyorum.
- Bir biti set, clear ve mask işlemleriyle değiştirebiliyorum.

- Timer clock, ARR, PSC, CCR ve duty ilişkisini biliyorum.
- Complementary PWM ile dead-time'ın neden farklı kavramlar olduğunu biliyorum.
- ISR'ın kısa tutulması ve paylaşılan verinin korunması gerektiğini biliyorum.
- Aşırı akım korumasında donanım yolunun zorunlu olduğunu biliyorum.

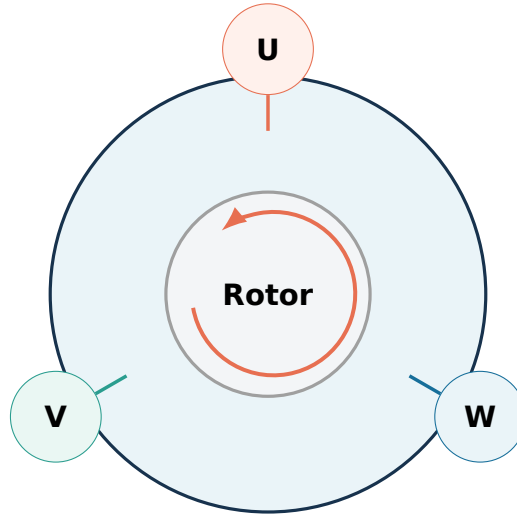
Kısım III

BLDC Motor Kontrolüne Geçiş

BLDC motorun temel çalışma mantığı

17.1 Stator, rotor ve üç faz

BLDC motorun statorunda U, V ve W olarak adlandırılan üç faz sargısı bulunur. Rotor kalıcı mıknatıs taşır. Sargılardaki akımı doğru sırada değiştirerek dönen manyetik alan oluştururuz; rotor bu alanı takip eder.



Şekil 17.1: Üç faz ve dönen mıknatıslı kısmın kavramsal görünümü

17.2 Elektriksel ve mekanik açı

Bir kutup çiftli motorda bir mekanik tur bir elektriksel tura karşılık gelir. p kutup çiftli motorda:

$$\theta_e = p \theta_m$$

Altı adımlı komütasyonda her sektör 60 elektriksel derecedir. 7 kutup çiftinde bir mekanik turda $6 \times 7 = 42$ sektör olayı görülür.

Neden?

“Motorun 60 derecelik aralıkları” ifadesi çoğunlukla milin her 60 mekanik derecesi anlamına gelmez. Komütasyon algoritmasındaki 60 derece elektriksel açıdır; kutup çifti sayısı mekanik karşılığını değiştirir.

17.3 Torku ne belirler?

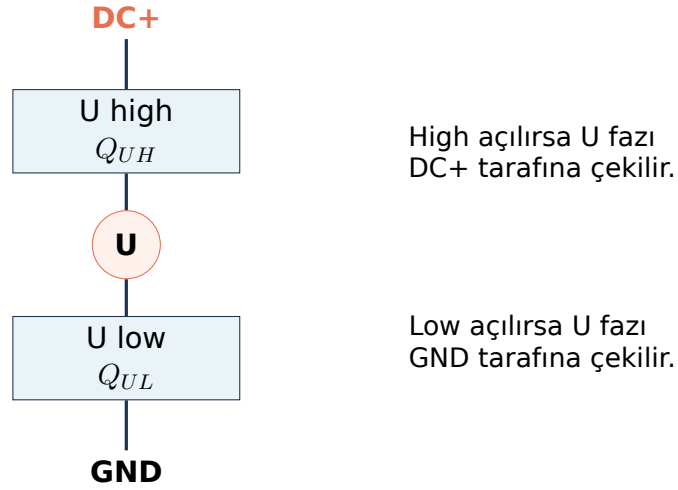
Basitleştirilmiş olarak motor torku faz akımıyla ilişkilidir. PWM duty artırıldığında uygulanan ortalama gerilim ve koşullara bağlı olarak akım artabilir. Ancak duty doğrudan tork değildir; sargı direnci, endüktans, hızla oluşan back-EMF, besleme gerilimi ve yük de etkilidir.

Bu nedenle “duty yüzde 50 ise hız kesin yüzde 50” veya “yüksek duty her zaman güvenlidir” denemez.

Üç faz inverter ve altı MOSFET

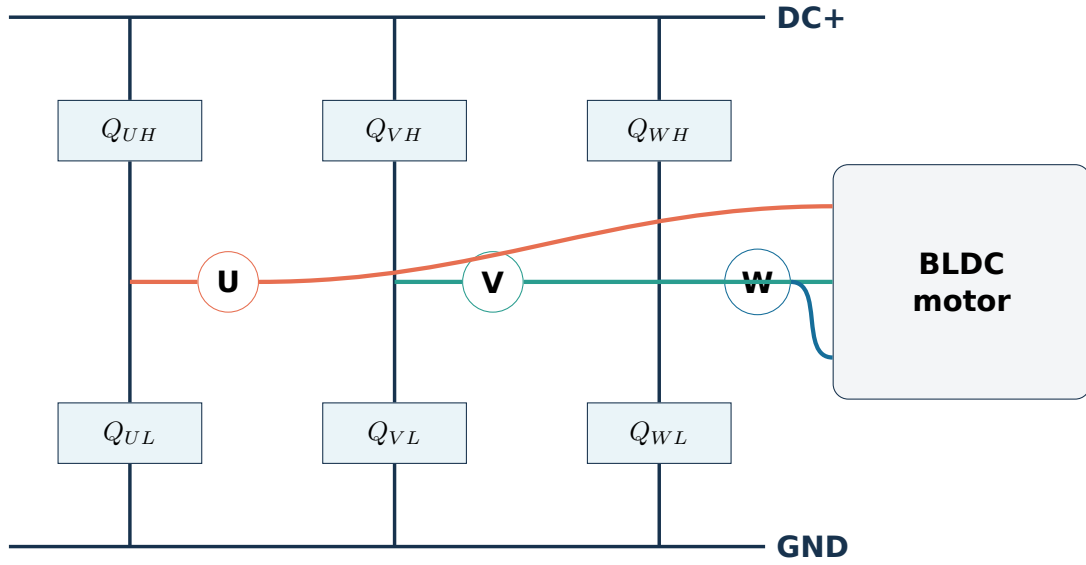
18.1 Bir yarım köprü

Her motor fazı bir high-side ve bir low-side MOSFET arasındaki ayrı orta noktaya bağlanır.



Şekil 18.1: U fazına ait tek yarım köprü

18.2 Üç yarım köprü bir arada



Şekil 18.2: U, V ve W orta noktaları birbirinden ayrı olan üç faz inverter

Şemada kritik ayrıntı

U, V ve W inverter çıkışları aynı kablo değildir ve birbirine kısa devre edilmez. Her biri kendi yarım köprüsünün orta noktasıdır ve motorun ayrı faz terminaline gider.

18.3 Üç fazın üç durumu

Bir komütasyon anında faz için kavramsal durumlar:

- **SOURCE:** High-side ile DC+ tarafına enerji verir.
- **SINK:** Low-side ile akımı GND tarafına taşır.
- **FLOAT:** İki anahtar da kapalıdır; sensorless yöntemde BEMF ölçülebilir.

Gerçek PWM stratejisinde "SOURCE" anahtarı sürekli açık olmak zorunda değildir; ilgili aktif anahtar PWM'lenebilir. Serbest dolaşım akımının yolu kullanılan synchronous/asynchronous rectification stratejisine göre değişir.

Altı adımlı komütasyon

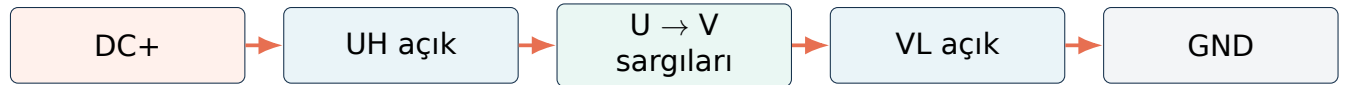
19.1 Neden altı adım?

Her adımda bir faz DC+ tarafında, bir faz GND tarafında, üçüncü faz boşta tutulur. Altı farklı yönlü faz çifti, elektriksel turu altı sektöre böler.

Adım	Kaynak	Sink	Boşta	Anahtarlar	Maske
1	U	V	W	UH + VL	0x09
2	U	W	V	UH + WL	0x21
3	V	W	U	VH + WL	0x24
4	V	U	W	VH + UL	0x06
5	W	U	V	WH + UL	0x12
6	W	V	U	WH + VL	0x18

19.2 Birinci adımı fiziksel okuyalım

Adım 1'de UH ve VL etkinleştirilir:



W fazı floating

Bit maskesi:

```

1 GATE_UH = 1u << 0; /* 000001 */
2 GATE_VL = 1u << 3; /* 001000 */
3
4 gate_mask = GATE_UH | GATE_VL; /* 001001 = 0x09 */
  
```

19.3 Yön nasıl ters çevrilir?

Komütasyon adımları ters sırada uygulanırsa dönen alanın yönü tersine döner. Ancak çalışan motorun yönünü aniden terslemek büyük akım ve mekanik darbe oluşturabilir. Önce kontrollü yavaşlatma veya durdurma gerekir.

19.4 Komütasyon zamanını kim belirler?

Üç temel yaklaşım vardır:

Yöntem	Bilgi kaynağı	Temel sorun
Open-loop	Yazılım belirli zaman aralığında ilerler	Yük altında rotor alanı takip etmeyebilir
Hall sensörlü	Üç dijital Hall kodu	Motor/kablo eşlemesi kalibre edilmelidir
Sensorless BEMF	Floating faz gerilimi	Düşük hızda BEMF zayıftır; başlangıç gerekir

Hall ve sensorless BEMF

20.1 IMU neden gerekli değildir?

IMU, doğrusal ivme ve açısal hız ölçer; klasik BLDC komütasyonunda gerekli sensör değildir. Rotorun elektriksel konumu Hall sensörü, enkoder veya fazlardan oluşan back-EMF ile tahmin edilir.

20.2 Hall kodları

Üç Hall hattı sekiz ikili kombinasyon üretebilir. Tipik olarak altı geçerli durum vardır; 000 ve 111 hata kabul edilir. Örnek ileri sıra:

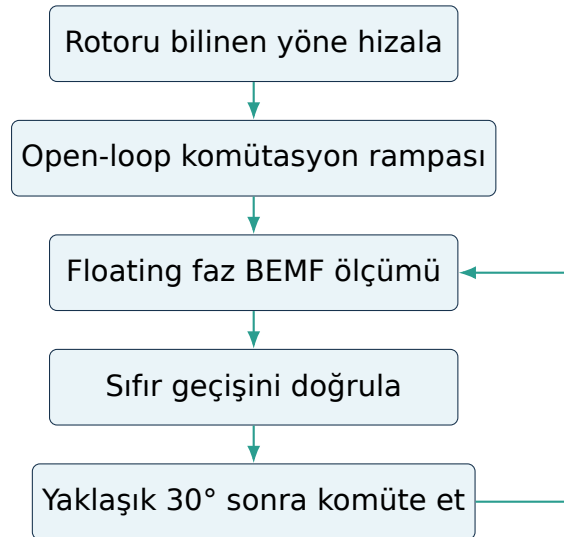
001 → 101 → 100 → 110 → 010 → 011

```
1 static const int8_t hall_to_step[8] =  
2 {  
3     -1, /* 000 */  
4     0, /* 001 */  
5     4, /* 010 */  
6     5, /* 011 */  
7     2, /* 100 */  
8     1, /* 101 */  
9     3, /* 110 */  
10    -1 /* 111 */  
11 };
```

Bu sıra evrensel değildir; Hall montajı ve U-V-W kablo sırası değişirse tablo değişir.

20.3 Sadece üç faz kablosu varsa

Motorundan yalnız U, V ve W çıkıyorsa dahili Hall bağlantısı yoktur. Sensorless altı adımın temel akışı:



Trapezoidal BLDC’de floating fazın BEMF’i sanal nötr/yarım DC bus seviyesini geçtiğinde yaklaşık sektör ortası, yani bir sonraki komütasyondan yaklaşık 30 elektriksel derece önce kabul edilebilir. Uygulamada blanking süresi, filtreleme, comparator/ADC gecikmesi ve hızla değişen zamanlama hesaba katılır.

Sensorless başlangıç

Motor dururken BEMF yoktur. Bu nedenle sıfır hızda yalnız BEMF okuyarak rotor konumu bulunamaz. Hizalama ve kontrollü open-loop hızlanma gerekir. Faz sırası veya rampanın eğimi yanlışsa motor yalnız titreyebilir ya da aşırı akım çekebilir.

20.4 60° olayından hız hesabı

Bir mekanik turda $6p$ sektör olayı vardır. Timer frekansı f_t , iki olay arasındaki tick sayısı N ise:

$$RPM = \frac{60f_t}{6pN}$$

Örneğin 1 MHz timer, 7 kutup çifti ve 2000 tick sektör süresinde:

$$RPM = \frac{60 \times 1\,000\,000}{6 \times 7 \times 2000} \approx 714$$

Sürücü kartını tanımadan kod yazma

“Motor sürücü” sözü birkaç farklı kartı ifade edebilir. İlk iş kart üzerindeki entegre ve pin isimlerini belirlemektir.

Kart tipi	MCU ne gönderir?	Komütasyonu kim yapar?
Hazır ESC	Servo PWM, DShot veya üretici protokolü	ESC’nin kendi işlemcisi
Entegre komütasyon sürücüsü	Hız/yön/enable veya SPI komutları	Sürücü entegresi
3-PWM inverter sürücüsü	Üç PWM ve kontrol pinleri	MCU + sürücü mantığı
6-PWM gate driver	UH/UL, VH/VL, WH/WL komutları	Büyük ölçüde MCU timer’ı
Yalnız gate driver	Gate sürme ve koruma; MOSFET’ler harici	MCU + harici güç katı

21.1 Veri sayfasında aranacak bilgiler

- Lojik giriş gerilimi STM32/ESP ile uyumlu mu?
- Giriş arayüzü 1-PWM, 3-PWM veya 6-PWM mi?
- Aktif-high/aktif-low girişler hangileri?
- Dahili dead-time ve shoot-through önleme var mı?
- Fault çıkışı open-drain mi, pull-up gerekiyor mu?
- Enable sırası ve bootstrap koşulları nedir?
- Akım ölçüm çıkışının kazancı ve ofseti nedir?
- Maksimum motor/bus gerilimi ve akımı nedir?

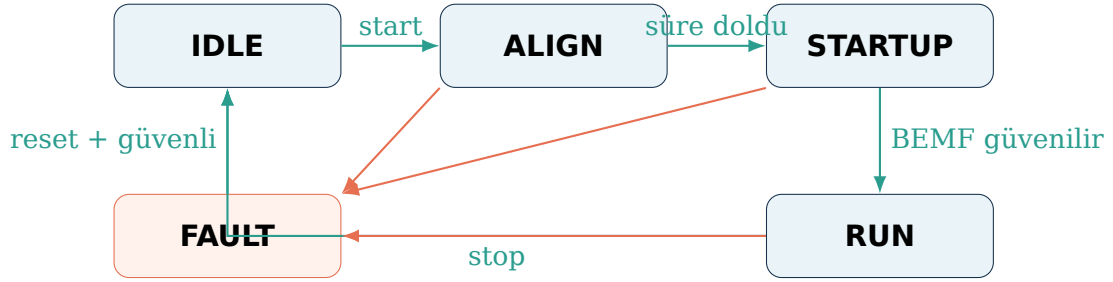
Dikkat

Kartın yalnız pin sayısına bakarak bağlantı yapılmaz. Aynı isimli girişler farklı sürücülerde farklı truth table’a sahip olabilir. Önce veri sayfası, ardından enerjisiz continuity kontrolü ve düşük gerilimli sınırlı akım testi yapılır.

Motor yazılımı için durum makinesi

22.1 Neden tek bir while döngüsü yetmez?

Motor bir anda “kapalı” durumdan normal sensorless çalışmaya geçemez. Başlatma, hizalama, rampalama ve hata gibi farklı davranışlar gerekir. Durum makinesi bu davranışları açıkça ayırır.



Şekil 22.1: Sensorless motor için örnek durum makinesi

22.2 Her durumun sorumluluğu

IDLE Bütün gate’ler güvenli kapalıdır; start komutu beklenir.

ALIGN Düşük duty ile sabit vektör uygulanır, rotor bilinen konuma çekilir.

STARTUP Önceden belirlenen zaman rampasıyla open-loop altı adım ilerletilir.

RUN Hall/BEMF olayına göre kapalı çevrim komütasyon ve hız kontrolü yapılır.

FAULT Gate’ler kapatılır, hata sebebi saklanır; koşulsuz otomatik yeniden başlatılmaz.

22.3 Durum geçiş koşulları

Bir durum geçişi yalnız zaman dolmasına bağlı olmamalıdır. Örnek RUN geçişi için:

- Minimum hız aşılmış mı?
- Art arda yeterli sayıda geçerli BEMF olayı görüldü mü?
- Akım ve bus gerilimi güvenli mi?
- Komütasyon sırası beklenen yönde mi?

Neden?

Durum makinesi, motorun “bazen çalışıyor bazen titriyor” davranışını gözlenebilir hale getirir. Debugger veya UART logunda hangi durumda kaldığını görerek sorunun hizalama, rampalama veya BEMF algılama tarafında olduğunu ayırabilirsin.

Kısım IV

Basitten Zora Kodlama Uygulamaları

Seviye 1: PC’de gate maskesi simülasyonu

Gerçek motorla başlamadan önce bit desenlerini PC’de üretmek en güvenli adımdır. Aşağıdaki program donanım sürmez; yalnız maskeyi hesaplar ve tehlikeli aynı-bacak desenini reddeder.

Listing 23.1: Tek dosyalık ilk simülasyon

```

1  #include <stdint.h>
2  #include <stdio.h>
3
4  enum
5  {
6      GATE_UH = 1u << 0,
7      GATE_UL = 1u << 1,
8      GATE_VH = 1u << 2,
9      GATE_VL = 1u << 3,
10     GATE_WH = 1u << 4,
11     GATE_WL = 1u << 5
12 };
13
14 static int has_shoot_through(uint32_t mask)
15 {
16     int u_bad = ((mask & GATE_UH) != 0u) &&
17                ((mask & GATE_UL) != 0u);
18
19     int v_bad = ((mask & GATE_VH) != 0u) &&
20                ((mask & GATE_VL) != 0u);
21
22     int w_bad = ((mask & GATE_WH) != 0u) &&
23                ((mask & GATE_WL) != 0u);
24
25     return u_bad || v_bad || w_bad;
26 }
27
28 int main(void)
29 {
30     uint32_t step_1 = GATE_UH | GATE_VL;
31
32     if (has_shoot_through(step_1) != 0)
33     {
34         printf("HATALI DESEN\n");
35         return 1;
36     }
37
38     printf("Adim 1 maskesi: 0x%02X\n",
39           (unsigned)step_1);

```

```
40     return 0;  
41 }
```

23.1 Satırların mantığı

1. Header'lar tip ve printf bildirimlerini getirir.
2. İsimsiz enum, gate bit sabitlerini gruplaştırır.
3. static yardımcı fonksiyonu dosyaya özel yapar.
4. mask & GATE_UH, UH bitinin maskede bulunup bulunmadığını ayırır.
5. != 0u, sonucu açık biçimde doğru/yanlış değerine çevirir.
6. &&, aynı bacağın iki anahtarının birlikte seçilip seçilmediğini sorar.
7. ||, üç fazdan herhangi birinde hata bulunmasını yeterli sayar.
8. GATE_UH | GATE_VL, iki ayrı biti tek maskede birleştirir.

23.2 Kendin dene

```
1 uint32_t bad = GATE_UH | GATE_UL;
```

Bu deseni kontrol fonksiyonuna gönderdiğinde hata üretmelidir. Böylece gerçek MOSFET olmadan koruma mantığını sınarsın.

Seviye 2: Komütasyon tablosu

24.1 Faz durumlarını tanımlamak

```

1 typedef enum
2 {
3     PHASE_FLOAT,
4     PHASE_SOURCE_HIGH,
5     PHASE_SINK_LOW
6 } PhaseMode;
7
8 typedef struct
9 {
10     PhaseMode u;
11     PhaseMode v;
12     PhaseMode w;
13 } CommutationStep;

```

Bu yapı sayesinde 0x09 gibi bir sayının anlamını akılda tutmak zorunda kalmayız. Önce fiziksel durumu ifade eder, ardından bit maskesine çeviririz.

24.2 Altı satırlık değişmez tablo

```

1 static const CommutationStep forward_table[6] =
2 {
3     {PHASE_SOURCE_HIGH, PHASE_SINK_LOW,    PHASE_FLOAT},
4     {PHASE_SOURCE_HIGH, PHASE_FLOAT,      PHASE_SINK_LOW},
5     {PHASE_FLOAT,       PHASE_SOURCE_HIGH, PHASE_SINK_LOW},
6     {PHASE_SINK_LOW,    PHASE_SOURCE_HIGH, PHASE_FLOAT},
7     {PHASE_SINK_LOW,    PHASE_FLOAT,      PHASE_SOURCE_HIGH},
8     {PHASE_FLOAT,       PHASE_SINK_LOW,    PHASE_SOURCE_HIGH}
9 };

```

24.3 Bir faz durumunu gate bitine çevirmek

```

1 static uint32_t phase_to_gate_bits(
2     PhaseMode mode,
3     uint32_t high_bit,
4     uint32_t low_bit)
5 {
6     switch (mode)
7     {

```



```

8      case PHASE_SOURCE_HIGH:
9          return high_bit;
10
11     case PHASE_SINK_LOW:
12         return low_bit;
13
14     case PHASE_FLOAT:
15     default:
16         return 0u;
17 }
18 }

```

Fonksiyon U, V veya W'yi özel olarak bilmez. Çağırın kod o fazın high ve low bitlerini parametre olarak verir. Bu, aynı mantığın üç kez kopyalanmasını önler.

```

1 static uint32_t step_to_gate_mask(
2     const CommutationStep *step)
3 {
4     uint32_t mask = 0u;
5
6     mask |= phase_to_gate_bits(
7         step->u, GATE_UH, GATE_UL);
8
9     mask |= phase_to_gate_bits(
10        step->v, GATE_VH, GATE_VL);
11
12    mask |= phase_to_gate_bits(
13        step->w, GATE_WH, GATE_WL);
14
15    return mask;
16 }

```

Sözdizimi

`const CommutationStep *step`, fonksiyonun yapıyı okuyacağını fakat bu işaretçi üzerinden değiştirmeyeceğini belirtir. `step->u`, işaret edilen yapının U üyesidir. `mask | = ...`, yeni gate bitlerini mevcut maskeye ekler.

Seviye 3: Fonksiyon işaretçisiyle donanım soyutlama

25.1 Problem

PC testinde gate yazmak ekrana sayı basmalıdır; STM32'de timer register'larını güncellemelidir. Kontrol algoritmasına iki ayrı sürüm yazmak istemeyiz.

25.2 Fonksiyon işaretçisi tipi

```
1 typedef void (*MotorWriteGatesFn)(uint32_t gate_mask);
2 typedef uint32_t (*MotorReadTicksFn)(void);
3
4 typedef struct
5 {
6     MotorWriteGatesFn write_gates;
7     MotorReadTicksFn read_ticks;
8 } MotorHwOps;
```

İlk typedef sağdan sola şöyle okunabilir:

MotorWriteGatesFn; bir uint32_t alan ve değer döndürmeyen fonksiyonu gösterebilen işaretçi tipidir.

25.3 PC uygulaması

```
1 static volatile uint32_t simulated_gate_register;
2
3 static void simulated_write_gates(uint32_t gate_mask)
4 {
5     simulated_gate_register = gate_mask;
6     printf("Gate register = 0x%02X\n",
7         (unsigned)simulated_gate_register);
8 }
9
10 static uint32_t simulated_read_ticks(void)
11 {
12     return simulated_tick_register;
13 }
14
15 static const MotorHwOps simulated_ops =
16 {
```

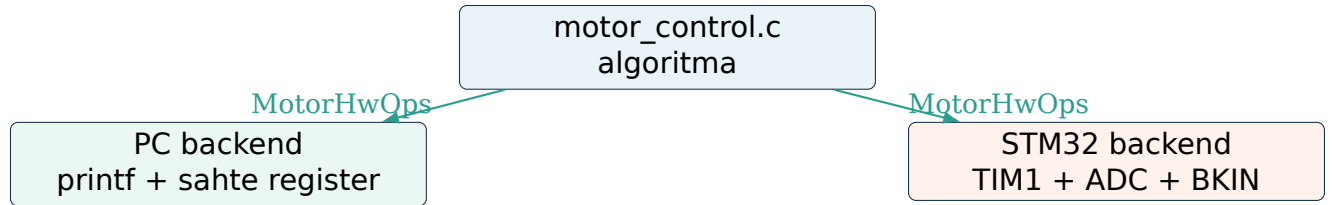
```
17     .write_gates = simulated_write_gates,  
18     .read_ticks = simulated_read_ticks  
19 };
```

25.4 Kontrol katmanının çağırısı

```
1 motor->hw->write_gates(gate_mask);
```

Bu satırda iki ok operatörü vardır:

1. `motor->hw`: Motor yapısındaki donanım arayüzü işaretçisini bulur.
2. `->write_gates`: Arayüz yapısındaki fonksiyon işaretçisini bulur.
3. `(gate_mask)`: İşaret edilen fonksiyonu çağırır.



Seviye 4: ISR posta kutusu ve olay işleme

26.1 Paylaşılan veri yapısı

```

1 typedef struct
2 {
3     volatile uint8_t hall_code;
4     volatile uint8_t event_pending;
5     volatile uint32_t capture_ticks;
6 } HallIsrMailbox;
7
8 static HallIsrMailbox hall_mailbox;
```

ISR yalnız veriyi yakalar:

```

1 void motor_control_hall_isr(
2     uint8_t hall_code,
3     uint32_t capture_ticks)
4 {
5     hall_mailbox.hall_code = hall_code & 0x07u;
6     hall_mailbox.capture_ticks = capture_ticks;
7     hall_mailbox.event_pending = 1u;
8 }
```

Ana döngü ağır işi yapar:

```

1 void motor_control_process(MotorControl *motor)
2 {
3     if ((motor == NULL) ||
4         (motor->state != MOTOR_RUN))
5     {
6         return;
7     }
8
9     if (hall_mailbox.event_pending == 0u)
10    {
11        return;
12    }
13
14    /* Gerçek MCU'da ıksa kritik bölüm gerekir. */
15    uint8_t hall = hall_mailbox.hall_code;
16    uint32_t capture = hall_mailbox.capture_ticks;
17    hall_mailbox.event_pending = 0u;
18
19    int8_t step_index = hall_to_step[hall];
20
21    if (step_index < 0)
```

```
22     {  
23         enter_fault(motor, FAULT_INVALID_HALL);  
24         return;  
25     }  
26  
27     update_speed(motor, capture);  
28     apply_step(motor, (uint8_t)step_index);  
29 }
```

26.2 Erken dönüşler neden kullanıldı?

Geçersiz koşulları fonksiyon başında eleyince ana başarılı akış daha az iç içe girer. Sıra bilinçlidir:

1. İşaretçi geçerli mi?
2. Motor RUN durumunda mı?
3. Yeni olay var mı?
4. Hall kodu geçerli adım üretiyor mu?
5. Sonra ölçüm ve komütasyon yapılır.

26.3 Bir event bayrağının sınırı

Tek event_pending bayrağı, ana döngü yavaşsa iki hızlı olayı tek olay gibi görebilir. Üretim kodunda olay kaybı kabul edilmiyorsa:

- monoton artan event counter,
 - ring buffer,
 - timer capture FIFO/DMA,
 - RTOS queue
- gibi mekanizmalar değerlendirilir.

Seviye 5: Hız hesabı ve hata yolu

27.1 İlk ölçüm neden ayrı ele alınır?

Hız için iki zaman damgası gerekir. Sağlam yaklaşım ayrı bir geçerlilik bayrağı kullanmaktır:

```

1 static void update_speed(
2     MotorControl *motor,
3     uint32_t capture_ticks)
4 {
5     if (motor->has_previous_capture == 0u)
6     {
7         motor->last_capture_ticks = capture_ticks;
8         motor->has_previous_capture = 1u;
9         return;
10    }
11
12    uint32_t sector_ticks =
13        capture_ticks - motor->last_capture_ticks;
14
15    motor->last_capture_ticks = capture_ticks;
16
17    if ((sector_ticks == 0u) ||
18        (motor->pole_pairs == 0u))
19    {
20        return;
21    }
22
23    uint64_t numerator =
24        60ull * motor->timer_frequency_hz;
25
26    uint64_t denominator =
27        6ull * motor->pole_pairs * sector_ticks;
28
29    motor->measured_rpm =
30        (uint32_t)(numerator / denominator);
31 }
```

Neden?

last_capture_ticks == 0 değerini “ilk ölçüm yok” işareti yapmak zayıftır; gerçek timer değeri meşru biçimde 0 olabilir. Ayrı has_previous_capture bayrağı durumu açıkça temsil eder.

27.2 Merkezi hata fonksiyonu

```
1 static void enter_fault(  
2     MotorControl *motor,  
3     MotorFault fault)  
4 {  
5     motor->fault = fault;  
6     motor->state = MOTOR_FAULT;  
7     motor->hw->write_gates(0u);  
8 }
```

Her hata dalının gate kapatmayı unutması yerine güvenli davranış tek fonksiyonda toplanır. Gerçek projede buna hata zaman damgası, ölçülen akım, bus gerilimi ve son sektör de eklenebilir.

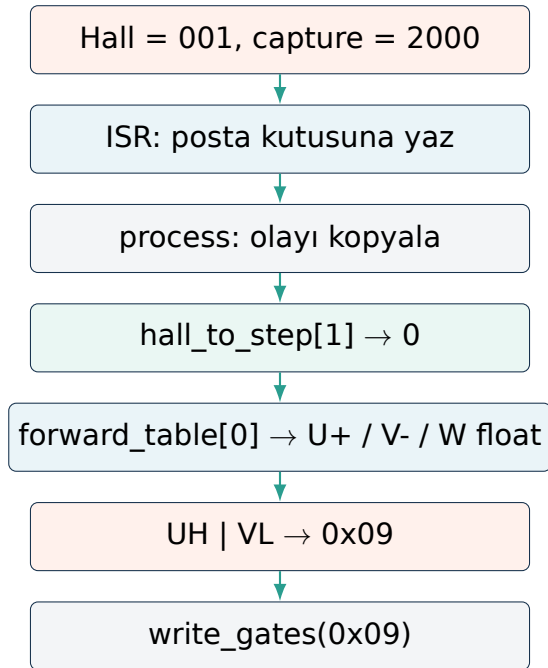
Tam programın çalışma sırası

28.1 main fonksiyonunun görevi

```
1 int main(void)
2 {
3     MotorControl motor;
4
5     motor_hw_init();
6     motor_control_init(
7         &motor,
8         motor_hw_get_ops(),
9         1000000u,
10        7u);
11
12     motor_control_start(&motor);
13
14     while (1)
15     {
16         motor_control_process(&motor);
17     }
18 }
```

1. Stack üzerinde MotorControl motor nesnesi oluşturulur.
2. GPIO, timer, ADC ve fault donanımı hazırlanır.
3. &motor ile aynı nesnenin adresi init fonksiyonuna verilir.
4. Timer frekansı ve kutup çifti sayısı kontrol bağlamına kaydedilir.
5. Start isteği state machine'i başlatır.
6. Sonsuz döngü bekleyen olayları ve durum geçişlerini işler.

28.2 Tek Hall olayının yolculuğu



28.3 Kodun kavram haritası

C kavramı	Kod örneği	Projede çözdüğü sorun
İşaretçi	MotorControl *motor	Aynı kontrol nesnesini fonksiyonlarda değiştirmek
volatile	ISR posta kutusu	Kesme/donanım kaynaklı değişikliği görünür kılmak
static	static void enter_fault	Modül içi ayrıntıyı gizlemek
const	Komütasyon tablosu	Kazara değişikliği önlemek
struct	MotorControl	İlişkili motor durumunu toplamak
enum	MotorState	Sayılarla anlamlı isim vermek
Bit işlemi	GATE_UH GATE_VL	Bir register/maskede çok anahtarı temsil etmek
Fonksiyon işaretçisi	MotorHwOps	Algoritmayı MCU'dan ayırmak
Header	motor_control.h	Public arayüzü bildirmek

STM32'ye taşıma rehberi

29.1 Hangi kod aynı kalır?

Genellikle şu kısımlar MCU'dan bağımsız tutulabilir:

- State machine ve fault mantığı
- Hall/BEMF sektör eşlemesi
- Komütasyon tablosu
- Hız hesabı
- Duty sınırlandırma ve üst seviye komutlar

Şu kısımlar hedefe özeldir:

- Clock tree ve pin alternate function ayarı
- TIM1/TIM8 register/HAL/LL çağrıları
- Complementary PWM, dead-time ve break konfigürasyonu
- ADC channel, trigger ve DMA ayarı
- Hall input capture veya comparator/EXTI ISR'ı

29.2 Kavramsal STM32 backend

```

1 static void stm32_write_gates(uint32_t mask)
2 {
3     /* Önce yeni compare ve channel-enable değerlerini
4        preload alanlarına hazırla. */
5
6     prepare_u_leg(mask);
7     prepare_v_leg(mask);
8     prepare_w_leg(mask);
9
10    /* Uygun timer ıolaynda hepsini birlikte şetkinletir. */
11    trigger_commutation_event();
12 }

```

Bu fonksiyonun içinde sıradan altı GPIO'yu art arda açıp kapatmak uygun değildir. Geçişlerin timer ile senkron, dead-time'lı ve break korumalı yapılması gerekir.

29.3 HAL, LL veya register?

Yaklaşım	Avantaj	Dezavantaj
HAL	Hızlı başlangıç, okunabilir API	Ek yük ve bazı gelişmiş zamanlama ayrıntılarında sınırlılık
LL	Register'a yakın, üretici desteği	HAL'dan daha fazla donanım bilgisi gerekir
Doğrudan register	Tam kontrol ve düşük ek yük	Taşınabilirlik düşük, hata riski ve doküman ihtiyacı yüksek

Başlangıç için CubeMX/HAL ile clock, GPIO ve temel timer kurulabilir; kritik komütasyon bölümünde LL veya kontrollü register erişimi gerekebilir. Kurumun mevcut sürücü mimarisi ve kod standardı önceliklidir.

29.4 ESP32 farkı

ESP32 ailesinde motor kontrolü için MCPWM benzeri çevre birimleri bulunabilir; kesin modül hedef modele göre değişir. Üst seviye algoritma aynı kalırken timer, dead-time, fault ve ADC sürücüleri ESP-IDF arayüzlerine taşınır.

Laboratuvar bring-up planı

Motoru ilk denemede doğrudan besleme ve yüksek duty ile çalıştırmak yerine aşamalı doğrulama yapılır.

30.1 Aşama 0: Doküman ve bağlantı

1. Motorun anma gerilimi, akımı, kutup çifti ve bağlantıları bulunur.
2. Sürücü kartının tam parça numarası ve giriş modu belirlenir.
3. MCU ile sürücü arasında ortak referans/izolasyon yapısı doğrulanır.
4. U, V, W çıkışlarının motor fazlarına gittiği kontrol edilir.
5. Fault, enable ve akım sense hatları belirlenir.

30.2 Aşama 1: Motor bağlı değilken

1. Gate komutları kapalı başlıyor mu?
2. PWM frekansı doğru mu?
3. High/low complementary mi?
4. Dead-time osiloskopta var mı?
5. Fault tetiklenince çıkış donanımsal kapanıyor mu?
6. Altı adım sırası düşük frekansta izlenebiliyor mu?

30.3 Aşama 2: Düşük enerji

1. Akım limitli ve mümkün olan düşük uygun bus gerilimi kullanılır.
2. Motor mekanik olarak güvenli sabitlenir, yük ayrılır.
3. Çok düşük duty ile hizalama denenir.
4. Faz sırası ve dönme yönü gözlenir.
5. Akım şekli ve sürücü sıcaklığı izlenir.

30.4 Aşama 3: Kapalı çevrime geçiş

1. Hall sırası veya BEMF olayları yalnız gözlenir ve loglanır.
2. Olayların komütasyon tablosuyla sırası doğrulanır.
3. Open-loop rampanın takip edildiği düşük hızda kanıtlanır.
4. Güvenilir olay sayısından sonra RUN'a geçilir.
5. Hız ve akım limitleri adım adım yükseltilir.

İlk enerji verme

Bu adımlar kurum laboratuvar prosedürünün yerine geçmez. Güç elektroniği konusunda yetkili bir mühendisin gözetimi, uygun sigorta, akım sınırlama, acil kapatma ve kişisel koruyucu ekipman gerekir.

Sık hatalar ve teşhis yolu

Belirti	Olası neden	İlk güvenli kontrol
Motor yalnız titriyor	Faz/Hall sırası yanlış; startup hızlı; duty yetersiz	Enerjiyi kes, sektör sırasını ve düşük hızlı dalgaları doğrula
Akım aniden yükseliyor	Shoot-through, yanlış faz deseni, kilitli rotor	Break/fault kaydını ve dead-time'ı motor bağlı değilken ölç
Yön ters	Faz sırası veya komütasyon yönü ters	Tablo yönünü kontrollü değiştir; çalışırken tersleme yapma
Hall 000/111	Kablo/pull-up/sensör besleme sorunu	Giriş seviyelerini multimetre/logic analyzer ile kontrol et
BEMF bulunamıyor	Hız düşük, floating faz yanlış, gürültü/blanking	Open-loop hızı, ölçüm penceresi ve sanal nötrü kontrol et
RPM çok yanlış	Kutup çifti, timer clock veya sektör katsayısı yanlış	Formülü ölçülen event frekansı ile elle doğrula
MCU reset oluyor	Besleme çökmesi, EMI, watchdog, HardFault	Reset cause register, besleme dalgası ve fault logunu incele
Kod optimize edilince bozuluyor	Eksik volatile, yarış durumu, tanımsız davranış	Uyarılar, ISR paylaşımı ve sınır ihlallerini incele

Mini alıştırmalar

Alıştırma 1: Bit maskesi

UH bit 0, VL bit 3 ise iki anahtarı seçen maskeyi ikilik ve hexadecimal yaz.

Alıştırma 2: İşaretçi

```
1 void stop_motor(MotorControl motor)
2 {
3     motor.state = MOTOR_IDLE;
4 }
```

Bu fonksiyon çağırının motor durumunu neden değiştirmez? İmzasını düzelt.

Alıştırma 3: volatile

ISR new_sample bayrağını 1 yapıyor, ana döngü okuyup sıfırlıyor. Bayrak neden volatile olmalı? Neden volatile bütün yarış sorunlarını çözmez?

Alıştırma 4: Timer

72 MHz timer clock'u, PSC =0 ve 18 kHz PWM için ARR yaklaşık kaç olmalıdır?

Alıştırma 5: Elektriksel açı

5 kutup çiftli motorda bir 60 elektriksel derece sektör kaç mekanik derecedir? Bir mekanik turda kaç sektör olayı vardır?

Alıştırma 6: Hata tasarımı

Geçersiz Hall kodunda yalnız return yapmak neden yetersizdir? Güvenli hata fonksiyonu hangi işlemleri yapmalıdır?

Alıştırma çözümleri

Çözüm 1

$$(1 \ll 0) | (1 \ll 3) = 000001 | 001000 = 001001_2 = 0x09$$

Çözüm 2

Parametre değer ile geçirildiği için yapının kopyası değişir. Adres gönderilmelidir:

```

1 void stop_motor(MotorControl *motor)
2 {
3     if (motor != NULL)
4     {
5         motor->state = MOTOR_IDLE;
6     }
7 }
```

Çözüm 3

volatile, ana döngünün bayrağı gerçekten bellekten tekrar okumasını ister. Ancak ISR'ın iki olayının tek bayrağa sıkışmasını, çok alanlı verinin tutarsız kopyalanmasını veya read-modify-write yarışını çözmez.

Çözüm 4

$$ARR + 1 = \frac{72\,000\,000}{18\,000} = 4000 \Rightarrow ARR = 3999$$

Çözüm 5

$$\theta_m = \frac{60}{5} = 12^\circ$$

Bir mekanik turda $6 \times 5 = 30$ sektör olayı vardır.

Çözüm 6

Yalnız dönmek önceki gate desenini aktif bırakabilir. Hata fonksiyonu en azından gate'leri kapatmalı, state'i FAULT yapmalı ve hata nedenini saklamalıdır. Donanım break yolu ayrıca bağımsız kapanma sağlamalıdır.

Son kontrol listesi ve çalışma planı

34.1 İlk hafta

- C tipleri, fonksiyonlar, diziler ve işaretçiler için küçük PC programları yaz.
- Bit maskesiyle altı gate desenini üret ve test et.
- struct, enum, static, const, volatile örneklerini debugger ile gözle.
- Çok dosyalı projeyi GCC ile derle.

34.2 İkinci hafta

- STM32/ESP clock, GPIO ve timer örneği kur.
- PWM'i önce LED/logic analyzer üzerinde doğrula.
- Input capture veya periyodik timer kesmesi yaz.
- ISR ile ana döngü arasında güvenli event aktarımı kur.

34.3 Üçüncü hafta

- Sürücü veri sayfasından pin ve giriş modunu çıkar.
- Motor bağlı değilken complementary PWM, dead-time ve fault kapatmayı ölç.
- Komütasyon tablosunu sürücü truth table'ına uyarlayıp simüle et.
- Akım ölçümünü kalibre et.

34.4 Dördüncü hafta

- Yetkili gözetiminde düşük enerjili open-loop test yap.
- Hall veya BEMF olaylarını logla ve sırayı doğrula.
- Hız hesabı, fault durumları ve restart politikasını test et.
- Son olarak duty/hız kontrolünü kontrollü artır.

Projeye hazır sayılman için

- Motorun neden MCU'ya doğrudan bağlanmadığını açıklayabilmelisin.
- Altı adımın her birinde akım yolunu çizebilmelisin.
- Bir gate maskesini bitlerden üretebilmelisin.
- Timer PWM, dead-time, break ve input capture kavramlarını ayırabilmelisin.
- Hall ile yalnız üç kablolu sensorless motorun yazılım farkını bilmelisin.
- ISR'ı kısa tutup ana işleme veri aktarabilmelisin.
- Geçersiz durumda gate'leri güvenli kapatan tek hata yolun olmalı.

Kısa sözlük

Terim	Kısa açıklama
ADC	Analog gerilimi sayısal değere çeviren çevre birimi.
Back-EMF / BEMF	Dönen motorun fazlarında oluşan ters elektromotor kuvvet.
BLDC	Elektronik komütasyonlu kalıcı mıknatıslı fırçasız DC motor.
Bootstrap	High-side gate sürmek için kullanılan şarj pompası/kondansatör yaklaşımı.
Break / BKIN	Timer PWM çıkışlarını hata anında donanımsal kapatan giriş.
Capture	Bir olay anındaki timer sayacını donanımın kaydetmesi.
Commutation	Faz akım yollarının rotor konumuna göre değiştirilmesi.
Dead-time	Aynı yarım köprüde iki anahtarın da kapalı tutulduğu kısa süre.
Duty cycle	PWM periyodunun aktif geçen oranı.
Gate driver	MCU lojik sinyalini MOSFET gate'ini uygun hız/seviyede sürececek hale getiren devre.
GPIO	Genel amaçlı dijital giriş/çıkış pini.
Hall sensor	Manyetik alanla rotor konum sektörü hakkında dijital bilgi veren sensör.
ISR	Kesme olduğunda çalışan kısa servis fonksiyonu.
Linker	Ayrı nesne dosyalarını ve adres yerleşimini tek firmware'de birleştiren araç.
MOSFET	İnverterde hızlı güç anahtarı olarak kullanılan transistör.
Open-loop	Geri bildirim doğrulanmadan önceden belirlenmiş komutla ilerleme.
Pole pair	Bir kuzey ve bir güney manyetik kutuptan oluşan çift.
PWM	Darbe genişliği modülasyonu; ortalama güç/gerilim kontrolü yöntemi.
Register	Donanım durum ve ayarlarının bit alanlarıyla tutulduğu özel konum.

Terim	Kısa açıklama
Shoot-through	Aynı yarım köprüde üst ve alt anahtarın birlikte ilettime girerek DC bus'ı kısa devre etmesi.
State machine	Sistemin sınırlı durumlar ve açık geçiş kurallarıyla modellenmesi.
Translation unit	Ön işlem sonrasında tek bir .c dosyası ve dahil ettiklerinin oluşturduğu derleme birimi.
Watchdog	Yazılım kilitlenirse sistemi sıfırlayabilen bağımsız zamanlayıcı.

Başvuru kaynakları

Gerçek kart üzerinde çalışırken genel internet örneğinden önce şu birincil dokümanlar kullanılmalıdır:

1. Seçilen STM32/ESP modelinin datasheet'i ve reference manual'ı.
2. Motor sürücü/gate driver entegresinin datasheet'i ve evaluation board şeması.
3. Motorun gerilim, akım, kutup çifti ve sensör bağlantı bilgileri.
4. Kartın gerçek şeması, pinout'u ve kurumun mevcut driver kodu.
5. ARM Cortex-M kullanılıyorsa ilgili ARM Architecture/Cortex-M generic user guide.
6. C dili için ISO C standardına uyumlu güvenilir başvuru kitabı ve derleyici kılavuzu.

Son hatırlatma

Bu dökümandaki register adresleri ve sürücü fonksiyonları kavramsaldır. Gerçek değerler kart seçilmeden belirlenemez. Datasheet'te doğrulanmayan bir pin modu veya gate deseni gerçek donanımda kullanılmamalıdır.